CPU scheduling

CPU and I/O bursts provide a model for comparing fairness, throughput, response, priorities, deadlines, and multiprocessor costs. A useful scheduling claim specifies both its workload and the guarantees it promises.

Prerequisites. Process states and queues, Context switching, Probability and expectation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Learning objectives

- Derive waiting, turnaround, and response times
- Compare FCFS, SJF, SRTF, RR, priority scheduling, and feedback queues
- Analyze preemption, starvation, priority inversion, and real-time schedulability

Central question. Under which workload and execution assumptions can a policy allocate finite CPU capacity correctly and predictably?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1. Basic concepts

- A process alternates between CPU execution and I/O waits. A CPU burst is computation between blocking events; scheduling may interrupt that burst without completing its CPU demand.
- The short-term scheduler selects a runnable kernel execution context. The dispatcher saves and restores context, performs required mode transitions, and resumes execution. Dispatch latency is overhead rather than application progress.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1.1. The scheduler's time scales

- Long-term admission controls which jobs enter and affects multiprogramming. Short-term scheduling selects the next execution context. Classical medium-term scheduling suspends or swaps work to reduce memory pressure.
- Admission changes the future workload. Batch throughput and interactive response therefore cannot be optimized by treating every decision as one CPU-selection policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1.2. Preemptive and nonpreemptive policies

- A nonpreemptive policy switches when the running task blocks, terminates, or yields. A preemptive policy can also switch on a quantum expiration or a newly ready higher-priority task.
- Preemption can improve responsiveness but adds state and run-queue work and permits more interleavings of shared kernel data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1. Model, ordering, and boundary cases

Model assumptions. Jobs have defined arrivals and CPU/I/O demands. Context can be saved and restored, and queue transitions are synchronized.

Invariant and ordering. Each modeled CPU runs at most one task. A task cannot be enqueued twice, run before arrival, or use an unreleased resource.

Counterexample or boundary. Ignoring switching cost can make an extremely short quantum look fair while useful CPU time collapses.

Worked check. With P1 burst 5 before I/O, P2 burst 2, one CPU and switch cost 1, FCFS runs P1[0,5), switch[5,6), P2[6,8). Useful-work fraction is 7/8=87.5% and switching overhead is 1/8=12.5%. The CPU is never idle, so busy-time CPU utilization is still 8/8=100%. These are different measures; omitting switching also incorrectly changes elapsed time from 8 to 7.

Reasoning prompt. Which event-order bug could both duplicate a queue entry and permit execution in the wrong state?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2. Scheduling criteria

- Utilization and throughput generally favor more completed useful work. Turnaround, waiting, and response favor lower delays. These objectives can conflict.
- Waiting time accumulates only in the ready state, excluding CPU service and blocked I/O. Response must be defined as first dispatch or first externally visible output; the two need not coincide.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2.1. Formulas and observation intervals

- Let arrival be a_i, first dispatch s_i, and completion f_i. Turnaround_i=f_i-a_i. Under the first-dispatch convention, response_i=s_i-a_i. Waiting_i is the sum of ready intervals, not generally turnaround minus CPU time when blocked I/O exists.
- A window that omits unfinished long jobs systematically favors short completions. Report the treatment of censored jobs rather than silently dropping them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2. Model, ordering, and boundary cases

Model assumptions. All jobs use the same time units, arrival definition, and completion criterion. The observation includes jobs still outstanding at its endpoint.

Invariant and ordering. Turnaround equals completion minus arrival. With compatible definitions, response does not exceed turnaround. Ready waiting excludes blocked time.

Counterexample or boundary. Maximizing utilization by indefinitely deferring short interactive work can violate response requirements.

Worked check. Arrival a=2, first CPU s=5, first visible output o=9, completion f=14, with [2,5) in the ready queue and no other ready waiting. Waiting=3, scheduling response=3, visible response=7, turnaround=12. They describe overlapping intervals and cannot be added.

Reasoning prompt. How can equal mean waiting coexist with very different maximum response?

Ready waiting and two response events

Let arrival be a_i, total ready intervals W_i, first CPU s_i, first visible output o_i, and completion f_i. Then

$$waiting_i = W_i, \quad turnaround_i = f_i - a_i, \quad schedule\ response_i = s_i - a_i, \quad visible\ response_i = o_i - a_i.$$

Blocked I/O is neither service nor ready waiting. Task A arrives at 0, is ready[0,2), runs[2,4), blocks[4,8), is ready[8,11), then runs[11,12). Turnaround=12, ready waiting=2+3=5, service=2+1=3, blocked=4. The decomposition 5+3+4=12 closes. Counting blocked time as waiting gives 9 and violates the definition.

Equal mean waiting need not imply equal maximum response. A and B can each accumulate 4 ready units, yet first run at 4 and 8 under different arrivals/blocking. First visible output can occur still later. Metric names, observed events, units, and percentiles must accompany comparisons.

6.3. Scheduling algorithms

- Define the comparison key, preemption rule, tie-break, and queue update. A lower mean wait does not establish a per-job bound.
- Nonpreemptive SJF minimizes mean wait for simultaneously available, known single CPU bursts on one CPU without switching cost. SRTF preempts according to remaining demand. Unknown bursts require estimates, so the ideal result no longer follows automatically.
- An exchange proof explains the SJF bound. For adjacent available jobs x>y, running x before y contributes x to their combined waiting; swapping contributes y. Later jobs see the same total x+y. Removing every inversion therefore cannot increase total waiting. Different arrivals or deliberate idling require a different argument.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.1. FCFS and the convoy effect

- FCFS serves arrival order without preemption. A long first job delays many short jobs and can create a convoy through CPU and I/O resources.
- With simultaneous arrivals and bursts 120,10,10 in that order, waits are 0,120,130. Their mean is 250/3, approximately 83.33 time units.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.2. SJF and SRTF

- SJF selects the shortest available next burst when the CPU becomes available. SRTF can preempt when a job with shorter remaining demand arrives.
- Continually arriving short jobs can starve a long job. An arbitrary tie-break is not a fairness guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.3. Round Robin

- RR rotates a ready queue using quantum q. A quantum large enough to finish every burst approaches FCFS; very small q increases timer and possible switching overhead.
- For arrivals zero, bursts 24/3/3, and q=4, the timeline is P1[0,4), P2[4,7), P3[7,10), P1[10,14), P1[14,18), P1[18,22), P1[22,26), P1[26,30). The total is 30. Once only P1 remains, a quantum boundary need not switch to a different thread.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3.4. Priorities and multilevel queues

- Priority policies can starve lower-priority work. Aging increases effective priority with waiting, but its rate, saturation, ties, and arrival assumptions determine whether waiting is bounded.
- Multilevel queues separate classes such as foreground and background. Feedback queues allow class migration according to behavior; quanta and promotion rules are part of the policy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3. Model, ordering, and boundary cases

Model assumptions. Arrivals, actual or estimated bursts, preemption, and ties are specified.

Invariant and ordering. Timeline intervals do not overlap. Each execution segment consumes no more than remaining demand, and arrivals and completions update the ready set.

Counterexample or boundary. A stream of underestimated short jobs can postpone a long job indefinitely.

Worked check. Simultaneous bursts 8/4/2 give FCFS waits 0/8/12, mean 20/3, and SJF execution order P3/P2/P1 with waits 0/2/6 in execution order, mean 8/3. RR q=2 gives completions P1/P2/P3=14/10/6; total useful execution remains 14.

Reasoning prompt. Which optimality statement must change if the burst ordering uses inaccurate predictions?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

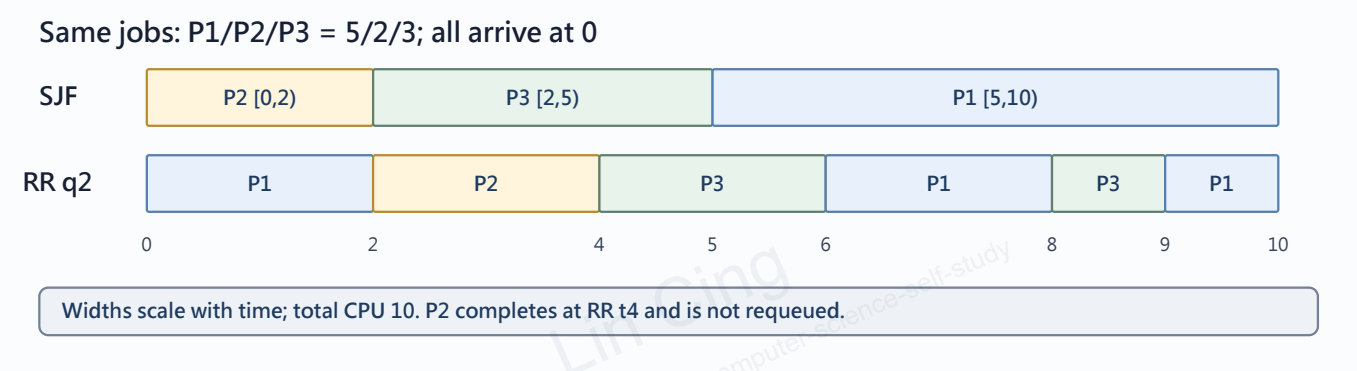
SJF and RR on the same 5/2/3 input

P1/P2/P3 all arrive at 0, bursts are 5/2/3, and switch cost is zero. SJF runs P2[0,2), P3[2,5), P1[5,10). In process order, completions are 10/2/5 and waits 5/0/2.

RR q=2 starts queue P1, P2, P3. It runs P1[0,2), P2[2,4), P3[4,6), P1[6,8), P3[8,9), P1[9,10). P2 completes at 4 and is never requeued. Completions are 10/4/9, waits 5/2/6, and first responses 0/2/4. Both consume 10 useful units; mean waits are 7/3 and 13/3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SJF and RR on the same 5/2/3 input (continued)



The separate 8/4/2 case instead has RR q=2 segments P1[0,2), P2[2,4), P3[4,6), P1[6,8), P2[8,10), P1[10,12), P1[12,14), and completions 14/10/6. Mixing those inputs changes the problem.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.4. Thread scheduling

- A library can schedule user threads within process-contention scope. Kernel-visible entities compete in system-contention scope.
- One-to-one mapping normally exposes each thread to kernel scheduling. In many-to-one, a genuinely blocking kernel call can stall the process's other user threads unless the runtime avoids that blocking.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.4. Model, ordering, and boundary cases

Model assumptions. The mapping and the runtime's handling of blocking calls are known.

Invariant and ordering. One kernel entity runs at most one context at a time. A runtime cannot promise independent progress without an available execution carrier.

Counterexample or boundary. The sole carrier blocking on I/O stops all other runnable tasks in the many-to-one example.

Worked check. T1 awaits network, T2 needs 4 CPU units. With one-to-one mapping T2 can run [0,4) while T1 blocks. With one carrier and a genuinely blocking T1 syscall, T2 has no guaranteed start, so completion at 4 is unjustified.

Reasoning prompt. Which mapping and blocking assumptions support a thread API's concurrency promise?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5. Multiprocessor scheduling

- Affinity can preserve cache and TLB locality. Excessive binding can leave CPUs idle while one queue is overloaded, so balancing trades locality against parallel use.
- Shared queues, migration, and work stealing must prevent a runnable thread from being dispatched on two CPUs simultaneously.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5. Model, ordering, and boundary cases

Model assumptions. Queue ownership and migration synchronization are defined, with cache and communication costs.

Invariant and ordering. A thread has only one actual running owner. Balancing respects required affinity constraints.

Counterexample or boundary. Binding every task to one core preserves some locality while wasting other cores.

Worked check. Fix eight cores, all jobs ready at t=0, indivisible jobs, and no job running on two cores at once. A/B each need 8 ms, C/D each need 4 ms, and E/F/G/H/I/J each need 2 ms: 36 ms of useful work. CPU0 initially queues [A,I,J]; CPU1 through CPU7 initially hold B,C,D,E,F,G,H respectively.

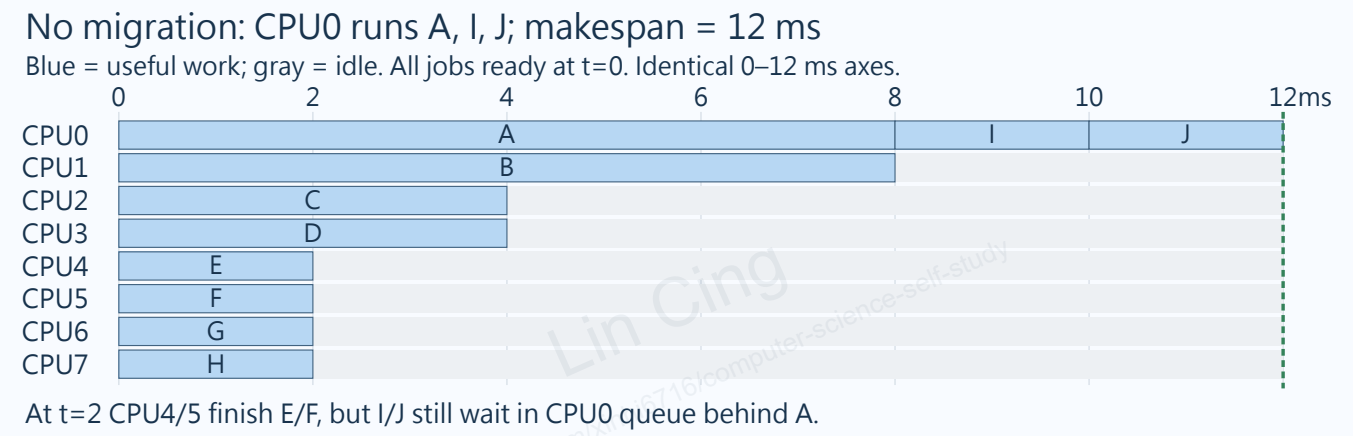
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5. Model, ordering, and boundary cases (continued)

Without migration CPU0 runs A[0,8), I[8,10), J[10,12), so makespan is 12 ms. At t=2 CPU4/5 finish E/F. Under queue synchronization they each remove one still-queued job I/J and transfer its unique ownership. Charge each migration 0.5 ms for queue/transfer and 0.5 ms for cache warmup. Both destination cores pay overhead [2,3), then run I/J [3,5). CPU0 finishes A at 8; makespan becomes 8 ms, speedup=12/8=1.5. Useful work remains 36 ms; added CPU work is 2 ms. These are specified model costs, not hardware measurements.

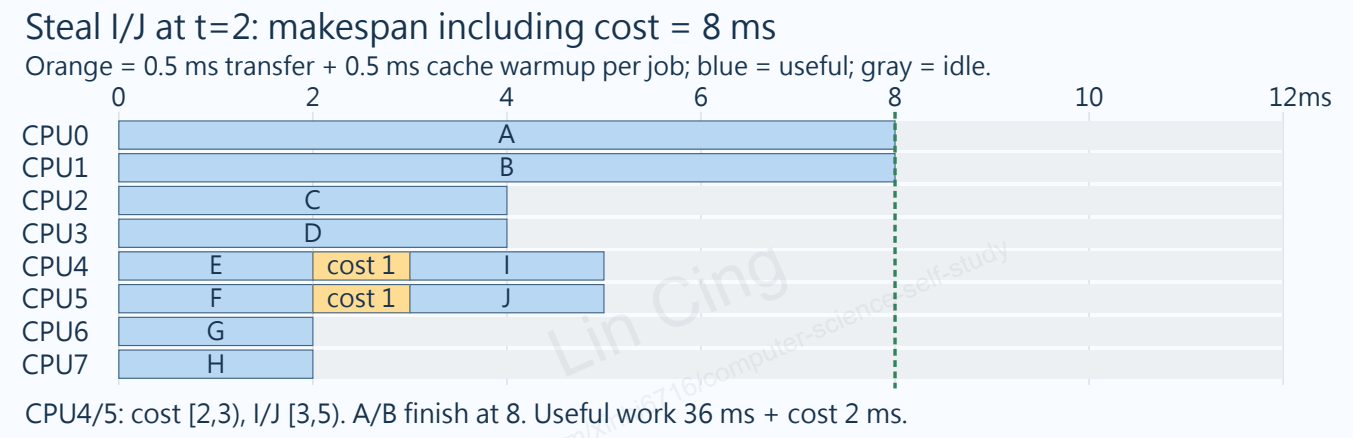
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5. Model, ordering, and boundary cases (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5. Model, ordering, and boundary cases (continued)



Indivisible A gives the lower bound $T \geq 8$, reached here. With per-migration cost c instead, makespan=max(8,2+c+2)=max(8,4+c). Strict improvement over 12 ms requires c<8. Dependencies, mandatory affinity, or different cache costs change the model; idle cores alone do not prove migration beneficial.

Real-time scheduling: distinguish tasks, jobs, and deadlines

A periodic task τ_i releases successive jobs. Let C_i be its execution-time upper bound, T_i its period, and D_i its relative deadline. With synchronous release at zero and job index k starting at zero,

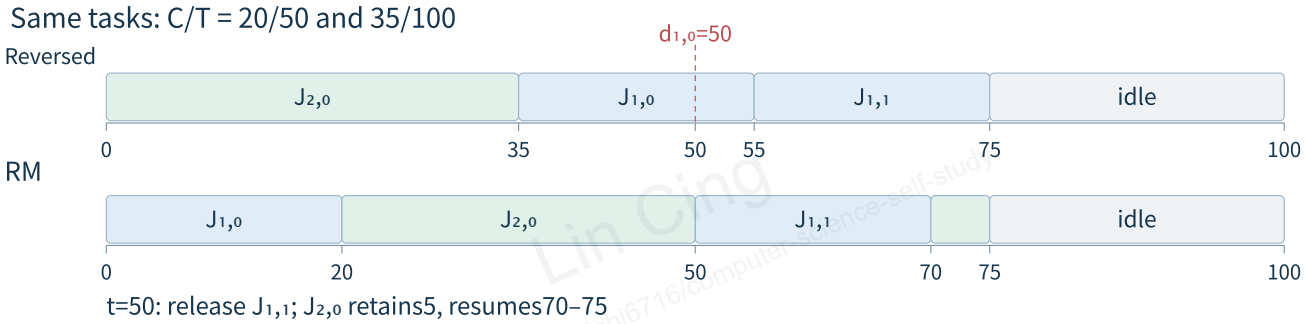
$$r_{i,k} = kT_i, \qquad d_{i,k} = r_{i,k} + D_i.$$

Base model: one CPU, immediate preemption, independent tasks, no lock blocking, I/O, self-suspension, switching or interrupt costs; $D_i=T_i$ except on the final page. A release adds a job to the ready set without necessarily changing the current job. Completion exactly at a deadline succeeds. At an event time t, account for execution up to t before processing new releases.

Quantity	Question answered	Distinction
C_i	How much CPU can each job require?	Not response time
T_i	How far apart are successive releases?	Not remaining time to deadline
D_i	How long after release may completion occur?	Absolute deadline is $r_{i,k}+D_i$
$U=\Sigma C_i/T_i$	What is the long-run CPU demand fraction?	Does not describe every short interval

RM gives shorter-period tasks higher fixed priority

Rate-monotonic (RM) assigns **higher fixed priority to a shorter period**, with preemption on a higher-priority release. Set τ_1=(C_1=20,T_1=D_1=50) and τ_2=(35,100,100); all times are milliseconds.



Reversed priorities delay $J_{1,0}$ to [35,55), missing50. Under RM, $J_{2,0}$ executes30 in [20,50), is preempted, then completes its **remaining five units** in [70,75).

RM completions20,70,75 meet deadlines50,100,100. At100 no old job remains and synchronous releases recur, proving repetition every100 ms.

The RM utilization bound is sufficient, not a rejection rule

For n independent preemptible tasks with D_i=T_i on the ideal single CPU, the classical **sufficient condition** is

$$U = \sum_i C_i/T_i \le n(2^{1/n} - 1).$$

For n=2 the bound is $2(\sqrt{2}-1) \approx 0.8284$. The preceding U=0.75 passes. The limit as n grows is ln $2 \approx 0.6931$. These are workload-independent guarantee bounds, **not a utilization ceiling for every RM workload**.

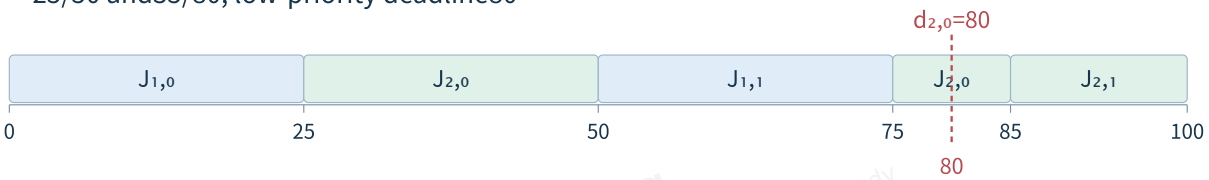
Counterexample: τ_1=(C=1,T=D=2), τ_2=(2,4,4) gives U=1. RM executes $J_{1,0}$ in [0,1), $J_{2,0}$ in [1,2), $J_{1,1}$ in [2,3), then the final unit of $J_{2,0}$ in [3,4). Completion times 1,3,4 meet deadlines 2,4,4. At t=4 the empty carry-in state and synchronous releases recur, so this schedule repeats.

Check assumptions, then apply the sufficient test. If it fails, use tighter interference analysis or a schedule whose recurring state is proved. U>0.8284 alone does not prove failure, and $U \le 1$ does not certify an arbitrary fixed-priority policy.

RM can miss a deadline even below full utilization

Now set τ_1=(25,50,50), τ_2=(35,80,80). Utilization is 15/16=0.9375. RM still gives τ_1 higher priority.

RM: C/T = 25/50 and35/80; low-priority deadline80



At80: old $J_{2,0}$ still needs5; new $J_{2,1}$ is separate

$J_{2,0}$ executes 25 units in [25,50), leaving ten. $J_{1,1}$ preempts it in [50,75). It receives only five more units in [75,80), so **five remain at deadline80**. The miss has already happened. If late jobs are retained, this job finishes at85. The new $J_{2,1}$ released at80 is a separate job.

Long-run spare capacity does not prevent fixed priorities from crowding a particular interval. The pending demand at80 proves failure; exceeding a sufficient utilization bound does not.

Count interference to check the RM completion time

For two synchronous tasks, τ_1 higher priority, $C_1 \leq T_1$ and no blocking, [0,R) contains $\lceil R/T_1 \rceil$ higher-priority releases at0,T_1,2T_1,… . **A release exactly at R is excluded.**

$$w^{(0)} = C_2, \qquad w^{(k+1)} = C_2 + \left\lceil w^{(k)}/T_1 \right\rceil C_1.$$

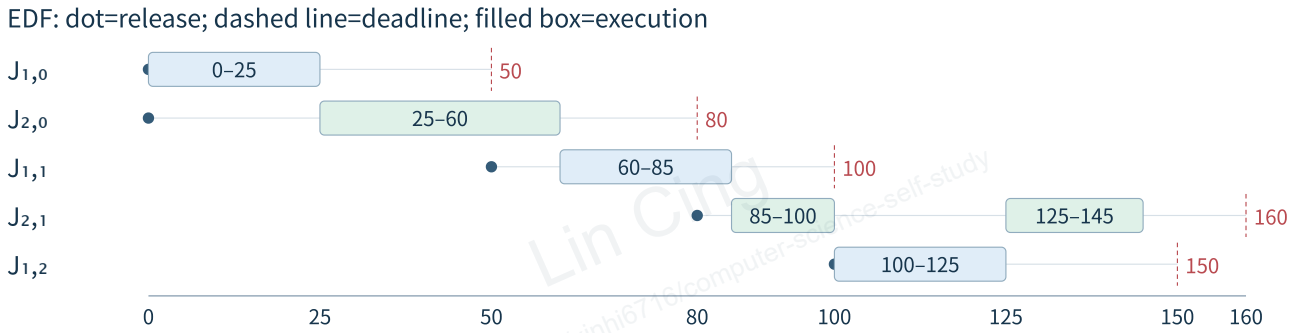
Iterate from C_2 to the earliest fixed point; stop if w>D_2.

Task pairs (C,T)	$w^{(0)}$	$w^{(1)}$	$w^{(2)}$	$w^{(3)}$	Check
(20,50),(35,100)	35	55	75	75	75≤100
(25,50),(35,80)	35	60	85	85	85>80
(1,2),(2,4)	2	3	4	4	Equality succeeds

Monotonicity makes iterates nondecreasing. Any fixed point $R^* \leq D_2$ bounds every iterate by induction; an iterate>D_2 excludes it. Only the first synchronous job is covered; other phases, tasks or blocking need further analysis.

EDF compares absolute deadlines of individual jobs

EDF selects the **earliest absolute deadline** among ready jobs. Use $\tau_1=(25,50,50),\tau_2=(35,80,80)$; ties retain current, otherwise task/job order.



At50, current $J_{2,0}$'s80 beats new $J_{1,1}$'s100: **continue to60**. At100, new $J_{1,2}$'s150 beats current $J_{2,1}$'s160: preempt.

The first150 ms alone cannot certify every future job; the theorem below does. Preemption preserves remaining execution.

Verify each EDF choice against the ready set

Each row is after processing that event and before executing the next interval. Parentheses contain (remaining CPU, absolute deadline); completed jobs are absent.

t (ms)	Ready jobs	Next execution	Reason
0	$J_{1,0}$(25,50),$J_{2,0}$(35,80)	$J_{1,0}$:0–25	50<80
25	$J_{2,0}$(35,80)	$J_{2,0}$:25–50	Only ready job
50	$J_{1,1}$(25,100),$J_{2,0}$(10,80)	$J_{2,0}$:50–60	80<100; continue
60	$J_{1,1}$(25,100)	$J_{1,1}$:60–80	Only ready job
80	$J_{1,1}$(5,100),$J_{2,1}$(35,160)	$J_{1,1}$:80–85	100<160; continue

At both50 and80 a new release changes the ready set while current remains earliest-due. Release alone does not imply preemption.

The same EDF rule does preempt at100

Continue with (remaining CPU, absolute deadline) for ready jobs.

t (ms)	Ready jobs	Next execution	Reason
85	$J_{2,1}$(35,160)	$J_{2,1}$:85–100	Only ready job
100	$J_{1,2}$(25,150),$J_{2,1}$(20,160)	$J_{1,2}$:100–125	150<160; preempt
125	$J_{2,1}$(20,160)	$J_{2,1}$:125–145	Only ready job
145	Empty	Idle:145–150	No ready job

The first three τ_1 completions are25,85,125 against50,100,150. The first two τ_2 completions are60,145 against80,160. **Release, priority reevaluation and switching are distinct events.** Releases at50 and80 do not change current.

Why does ideal EDF need only U≤1 when D=T?

Use the stated one-CPU model, starting without carry-in at t=0; tasks may have different nonnegative release phases. If U>1, demand due by time t is Ut−O(1). It eventually exceeds the available t, so no policy can meet all deadlines forever.

For sufficiency suppose U≤1, yet EDF first misses a deadline b. Consider only jobs with deadlines at most b. Let [a,b) be their last continuously backlogged interval, of length L=b−a. None of these jobs was pending immediately before a, so the relevant jobs all release at or after a. Throughout the interval EDF executes only such jobs.

Since $D_i=T_i$, at most $\lfloor L/T_i \rfloor$ jobs of task i can both release and become due in this interval: the first cannot be due before $a+T_i$ and later deadlines are separated by T_i. Thus their total demand satisfies

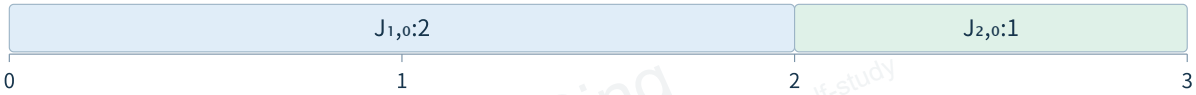
$$\sum_i N_i C_i \le \sum_i \lfloor L/T_i \rfloor \, C_i \le L \sum_i C_i/T_i = LU \le L.$$

A miss after L units of uninterrupted service instead requires demand **greater than L**, a contradiction. Hence U≤1 is sufficient and, with the necessity argument, exact in this model. The proof bounds every relevant interval rather than relying on an average alone.

Shorter relative deadlines can defeat low utilization

Let both tasks have C=2,T=10,D=3, released together at0. U=0.4, yet four CPU units must complete by3. A single CPU supplies only three in [0,3). **Every policy, including EDF, must miss at least one deadline.**

C=2,T=10,D=3: both release0; four units required by3



Capacity3<demand4; $J_{2,0}$ still needs1 at3

For $D_i \leq T_i$ the maximum demand both released and due within length L is

$$\mathrm{dbf}_i(L) = \max\left(0, \left\lfloor \frac{L - D_i}{T_i} \right\rfloor + 1\right) C_i.$$

The first job needs D_i time; each additional T_i permits another. At L=3 total demand4>3 proves impossibility. Passing just one length cannot certify all intervals. Priority inheritance handles specific lock-induced inversion; it cannot fix this capacity contradiction.

Historical scheduler mechanisms

Solaris converts class priorities to global priorities. TS/IA uses feedback; priority 0 has a 200 ms quantum, priority 59 a 20 ms quantum. CPU exhaustion and sleep return use different new priorities. Windows 7 uses 32 ready queues with versioned boosts. These are historical mechanisms rather than present-day specifications.

Linux 3.2 CFS stores ready entities in a red-black tree keyed by vruntime; current is outside the tree. The increment is actual execution×1024/weight. Nice 0 has weight 1024; nice 5 has 335. Select the smallest vruntime, subject to the version's bounded buddy preferences.

Historical scheduler mechanisms (continued)

Same workload, three ready structures

Solaris TS/IA

global priority

p20 → q120 ms
sleep return → p50
highest global priority
equal priority: round robin
IA favors window apps

Windows 7

32 ready queues

p1–15 variable / p16–31 RT
time quantum expires → lower
I/O / foreground → boost
foreground quantum ≈ 3×
scan highest nonempty queue

Linux 3.2 CFS

red-black tree

key = vruntime
nice 0/5 weight 1024/335
2 runnable → 3 ms slice
tick: delta_exec > slice
HRTICK queues exact slice

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed Linux 3.2 event model

One CPU, P1 burst 8 ms, P2 burst 4 ms, both runnable at 0. P3 burst 2 ms sleeps until 9. All have nice=0, weight=1024, and normalized initial vruntime=0. P1 enters the tree first on equal keys. Fix GENTLE_FAIR_SLEEPERS=true, wakeup granularity=1 ms, CONFIG_SCHED_HRTICK=y. Switch cost, interrupt cost, and timer jitter are zero. Normalization does not claim a real boot-time START_DEBIT offset; one common shift preserves relative comparisons.

For n runnable entities,

$$period = \begin{cases} 6\ \text{ms}, & n \le 8, \\ 0.75n\ \text{ms}, & n > 8, \end{cases} \qquad ideal_i = period \frac{weight_i}{\sum_j weight_j}.$$

Ideal runtime is a wall-time slice, not a global quantum. Two equal tasks get 3 ms each; three get 2 ms each. Current leaves the ready tree and returns with updated vruntime on preemption; blocked/completed tasks are not ready. HRTICK, a satisfied wakeup condition, or completion must justify a switch. No event at t=8 justifies splitting P1's [6,9) slice.

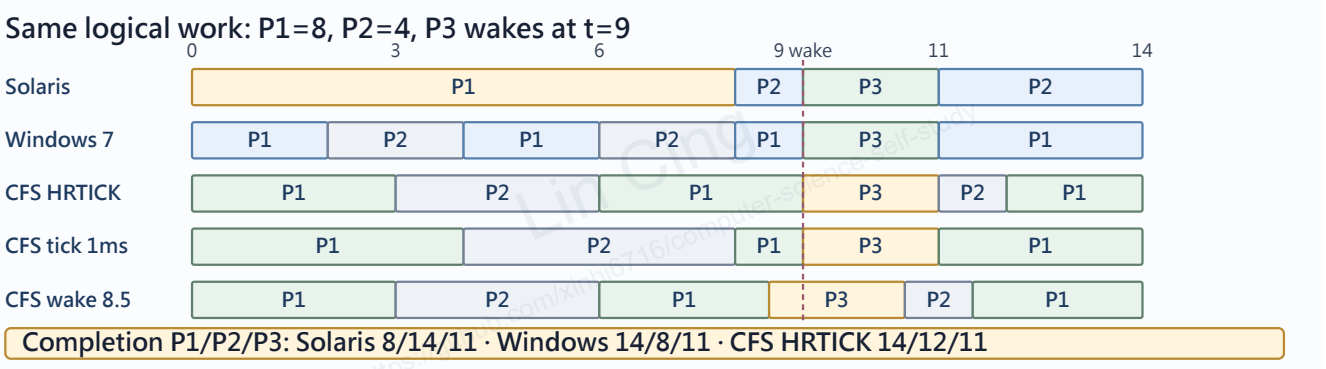
CFS event trace

States below are after the event and next-current selection. Ready excludes current; min_vruntime never moves backward.

Time/event	Current	Other ready	min_vruntime	New ideal	Reason
0 initial	P1	P2@0	0	3	Equal keys, P1 inserted first
3 HRTICK	P2	P1@3	0	3	P1 used slice
6 HRTICK	P1	P2@3	3	3	Equal keys, P1 already leftmost
9 HRTICK/wake	P3	P2@3, P1@6	3	2	P3 placed at 0, lead 6>1
11 P3 complete	P2	P1@6	3	3	P2 key 3 precedes 6
12 P2 complete	P1	None	6	6	Only P1, remaining 2
14 P1 complete	None	None	8	None	No work remains

CFS event trace (continued)

P3 placement at 9 is $\max(v_{sleep}, min_vruntime - 3) = \max(0, 3 - 3) = 0$. P1's key is 6. Wakeup compares current−wakeup>1 ms, so 6−0>1 and P3 immediately runs at 9, not earlier. Gantt is P1[0,3), P2[3,6), P1[6,9), P3[9,11), P2[11,12), P1[12,14). Completions are 14/12/11, not 14/9/11.



Tick, HRTICK, and wakeup sensitivity

Settings	Gantt	Completion P1/P2/P3
HRTICK, P3 wake 9	P1 0–3, P2 3–6, P1 6–9, P3 9–11, P2 11–12, P1 12–14	14/12/11
No HRTICK, 1 ms tick, wake 9	P1 0–4, P2 4–8, P1 8–9, P3 9–11, P1 11–14	14/8/11
HRTICK, wake 8.5	P1 0–3, P2 3–6, P1 6–8.5, P3 8.5–10.5, P2 10.5–11.5, P1 11.5–14	14/11.5/10.5

check_preempt_tick uses strict delta_exec>ideal_runtime. At t=3, 3>3 is false; the ordinary tick first preempts at t=4. Queued HRTICK instead directly requests rescheduling at the programmed slice boundary, without reapplying that strict comparison. Without HRTICK, P2 finishes at 8. At t=9 min_vruntime=5, so P3 returns at max(0,5−3)=2; P1 is 5, and 5−2>1 gives the wake preemption.

Tick, HRTICK, and wakeup sensitivity (continued)

The periodic tick also checks, after minimum granularity, whether current leads the tree's leftmost by more than ideal runtime. This handles fairness discrepancies but does not replace wakeup comparison. Last/next/skip buddy are locality/scheduling hints subject to virtual-time fairness. In this example n<8 before wake, so there is no ordinary next buddy from the sleeping P3; a successful wakeup comparison can nominate it. A buddy too far ahead cannot override the fairness bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nice weights and historical comparisons

With only A nice 0 and B nice 5 runnable, total weight=1359 and period=6 ms.

Task	Weight	Ideal runtime	vruntime per real ms	CPU share
A	1024	$6\times1024/1359\approx4.521$ ms	1 ms	75.35%
B	335	$6\times335/1359\approx1.479$ ms	$1024/335\approx3.057$ ms	24.65%

Weights are not linear nice percentages. Above eight runnable tasks the period grows to preserve minimum granularity instead of shrinking slices without bound.

Fixed case	Rules	Logical workload completion P1/P2/P3
Solaris TS/IA	p20/q120 ms, sleep return p50	8/14/11
Windows 7	32 queues, normalized 2/6 ticks, I/O boost	14/8/11
Linux 3.2 CFS	HRTICK, nice 0, 6 ms period, P3 wake 9	14/12/11

Nice weights and historical comparisons (continued)

These are specified event models, not product benchmarks. The same workload alone does not determine one Gantt. Timer mode, wake time, insertion ties, and placement are inputs; changing them yields the alternate completions above.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8. Algorithm evaluation

- Deterministic models calculate a trace exactly. Queueing models use arrival and service distributions. Simulation supports more complex policies. An implementation captures actual hardware and OS costs.
- Disclose workload distribution, processors, switching, and observation window. Means can hide starvation and long tails.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8. Model, ordering, and boundary cases

Model assumptions. Methods use comparable inputs and costs. Random simulations retain seeds and an appropriate uncertainty analysis.

Invariant and ordering. If models disagree on the same simplified workload, investigate omitted costs and distribution assumptions. Agreement is not itself an algorithmic invariant.

Counterexample or boundary. One deterministic SJF success does not establish superiority under every burst-estimation error distribution.

Worked check. Use an exactly replayable deterministic comparison: one CPU; P1/P2/P3 arrive at 0 with bursts 5/2/3 ms; no I/O or switching cost. Response means time to first CPU. Define p99 by nearest rank: with three observations, rank ceil(0.99×3)=3 is this batch maximum. Waiting=completion−burst.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8. Model, ordering, and boundary cases (continued)

Policy	Completion P1/P2/P3	Waiting P1/P2/P3	Mean waiting	First response P1/P2/P3	Batch p99 response
FCFS, initial P1/P2/P3	5/7/10	0/5/7	4	0/5/7	7
SJF, P2/P3/P1	10/2/5	5/0/2	7/3	5/0/2	5
RR q=2, initial P1/P2/P3	10/4/9	5/2/6	13/3	0/2/4	4

All use window [0,10], so throughput=3/10 jobs/ms. SJF has smaller mean waiting while RR has smaller batch first-response tail. Neither metric substitutes for the other. These are exact values for three specified jobs, not random simulation or population-p99 estimates; no 95% confidence interval is claimed. Random experiments require arrival/service distributions, seeds, independent replication units, and an estimator. A confidence interval for a mean is not a range containing 95% of individual waits.

6.8. Model, ordering, and boundary cases (continued)

Reasoning prompt. How would an evaluation expose the tail rather than only mean throughput?

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.9. Summary

- Scheduling correctness includes queue ownership and no double execution. Service quality adds explicitly required waiting bounds and response distributions.
- High CPU utilization does not imply short user delay. Additional runnable work can increase shared-resource contention and reduce useful throughput.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.9. Model, ordering, and boundary cases

Model assumptions. Correctness, quality, and cost receive separate evidence.

Invariant and ordering. If starvation freedom is required, establish it under admission and arrival assumptions. Fixed priorities alone may not provide it.

Counterexample or boundary. A 99% busy batch CPU that indefinitely postpones interactive work fails an interactive-service requirement.

Worked check. Retain RR q=2, all arrivals 0, and initial order P1/P2/P3; multiply every burst by k. Pop the queue head, run min(2,remaining), and append it only if unfinished. Exact replay gives:

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.9. Model, ordering, and boundary cases (continued)

k; bursts	Completion P1/P2/P3	Waiting P1/P2/P3	Throughput jobs/ms	Batch p99 first-CPU response	Maximum waiting
1; 5/2/3	10/4/9	5/2/6	3/10	4	6
2; 10/4/6	20/10/16	10/6/10	3/20	4	10
3; 15/6/9	30/16/25	15/10/16	1/10	4	16

For k=2 the complete trace is P1[0,2), P2[2,4), P3[4,6), P1[6,8), P2[8,10), P3[10,12), P1[12,14), P3[14,16), P1[16,18), P1[18,20). First responses remain 0/2/4 and busy utilization remains 100%, yet maximum ready waiting grows 6→10→16. First-CPU response, throughput, and waiting measure different effects. A finite batch does not prove starvation freedom under unbounded arrivals.

Reasoning prompt. Which per-task fairness measure reveals starvation hidden by aggregate utilization?

Chapter conclusions

- Scheduling names require arrival and burst assumptions
- Response and turnaround are different measures
- Multiprocessor and real-time analysis must specify locality, deadlines, and failure assumptions

Exercises and sources

- the separate exercises PDF.
- the separate solutions PDF.
- Coverage and source limitations.

Reference: A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th edition, Wiley, 2013. These are original explanations and original exercises. Historical product cases refer to the historical versions in the edition. No publisher solution text is used.

- Linux v3.2 sched_fair. c defines period/slice, sleep placement, tick, HRTICK, and wakeup comparisons.
- Linux v3.2 sched. c weight table.
- Microsoft priority boosts and Solaris time-sharing class.

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study